FMX Academy

INFORMAÇÕES			
MÓDULO		**AULA**	**INSTRUTOR**
07	**ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance**	**01 - Domínios Ricos vs. Domínios Anêmicos**	**Guilherme Paracatu**

1. Fundamentação Teórica

OBJETIVO: Entender por que a lógica de negócio deve morar dentro da entidade e como isso protege nosso sistema de bugs catastróficos.

O que é um Domínio Anêmico?

É um modelo onde as entidades são apenas **estruturas de dados passivas**. Elas possuem propriedades públicas (Get/Set), mas **zero lógica**.

- **Onde fica a lógica?** Espalhada em Services, Handlers ou até nos Controllers.
- **O problema:** A entidade não se protege. Você pode criar um Veiculo sem placa ou com ano "9999" e ela não vai reclamar. O sistema se torna um "caos" de validações repetidas em todo lugar.

O que é um Domínio Rico?

É o coração do **DDD (Domain-Driven Design)**. Aqui, a entidade é a dona das suas regras.

- **Encapsulamento:** As propriedades têm private set. Ninguém de fora muda o estado do objeto sem passar por uma regra.
- **Integridade (Invariantes):** O objeto garante que ele nunca nasça ou permaneça em um estado inválido.
- **Comportamento:** Se um veículo precisa "Trocar Placa", ele tem um método para isso que valida a nova placa antes de alterar.

Analogia: Um Domínio Anêmico é como um boneco de ventríloquo (só se mexe se alguém puxar a corda). Um Domínio Rico é como uma pessoa autônoma (ela decide se pode ou não realizar uma ação baseada em suas regras internas).

🛠️ **Parte 2: Prática e Comparação**
Vamos ver a diferença real no código usando o exemplo que estamos construindo na **ITB.**

❌ **Exemplo A: O Jeito Antigo (Domínio Anêmico)**
Neste modelo, a classe é "burra". Qualquer um pode estragar os dados.

Código C#
```
// ITB.Domain/Entities/Veiculo.cs
public class Veiculo
{
    public int Id { get; set; }
    public string Modelo { get; set; }
    public string Placa { get; set; }
    public int Ano { get; set; }
}

// O problema aparece no Handler/Service:
public async Task Handle(AdicionarVeiculoCommand command)
{
    var veiculo = new Veiculo();
    veiculo.Modelo = command.Modelo;
    veiculo.Placa = ""; // O código aceita placa vazia!
    veiculo.Ano = 1800; // O código aceita ano inválido!

    // A lógica de validação fica "jogada" aqui fora,
    // e se outro programador criar um veículo em outro lugar, ele vai esquecer de
validar.
    await _repository.Add(veiculo);
}
```

☑️ **Exemplo B: O Novo Jeito (Domínio Rico)**
Agora, a entidade se protege. Usamos **Cláusulas de Guarda (Guards)** no construtor.

Código C#
```
// ITB.Domain/Entities/Veiculo.cs
public class Veiculo
{
    public int Id { get; private set; } // Ninguém de fora altera o Id
    public string Modelo { get; private set; }
    public string Placa { get; private set; }
    public int Ano { get; private set; }

    // Única porta de entrada para criar um Veículo
    public Veiculo(string modelo, string placa, int ano)
    {
```

```
        // Regras de Integridade (Guards)
        if (string.IsNullOrWhiteSpace(modelo))
            throw new DomainException("Modelo é obrigatório.");

        if (placa.Length != 7)
            throw new DomainException("Placa inválida.");

        if (ano < 1900 || ano > DateTime.Now.Year + 1)
            throw new DomainException("Ano do veículo fora da realidade.");

        Modelo = modelo;
        Placa = placa;
        Ano = ano;
    }

    // Comportamento: Alterar a placa requer validação
    public void AlterarPlaca(string novaPlaca)
    {
        if (string.IsNullOrWhiteSpace(novaPlaca)) throw new DomainException("Nova placa
inválida.");
        Placa = novaPlaca;
    }
}
```

⚖️ **Comparativo:**

Característica	Domínio Anêmico (O que fazíamos)	Domínio Rico (O que faremos)
Propriedades	public get; public set;	public get; private set;
Validação	No Handler ou Controller (Espalhada)	No Construtor da Entidade (Centralizada)
Segurança	Baixa (Objeto pode ser criado vazio)	Alta (Objeto nasce sempre válido)
Manutenção	Difícil (Regras repetidas em vários lugares)	Fácil (Regra de negócio em um só lugar)
Orientação a Objetos	Usa a classe apenas como "pasta"	Usa a classe como um Objeto Real

🎯 **Por que isso muda o jogo?**

Ao usarmos **Domínios Ricos**, garantimos que mesmo que o desenvolvedor esqueça de validar no FluentValidation (entrada), o banco de dados nunca será corrompido, porque a classe Veiculo simplesmente não "aceita" nascer se os dados estiverem errados.

2. Como o EF Core lê Domínios Ricos

A "Mágica" do EF Core (Teoria)

O Entity Framework Core não usa os mesmos caminhos que um programador comum. Para garantir que um Domínio Rico funcione, ele utiliza três mecanismos:

1. **Backing Fields (Campos de Apoio):** O EF consegue ler e escrever diretamente nos campos privados da classe, ignorando o private set.

2. **Construtores sem parâmetros (Protected/Private):** O EF precisa de um construtor "vazio" para criar a instância antes de preencher os dados. Podemos deixar esse construtor como protected para que ninguém o use por erro, mas o EF consiga acessá-lo.

3. **Mapeamento por Reflexão:** O framework utiliza reflexão para entender a estrutura da classe, mesmo com o encapsulamento rígido.

🛠️ **Parte 3: Prática e Comparação (Configuração)**
Vamos ajustar o nosso código para que o Domínio Rico seja 100% compatível com o banco de dados.

❌ **O que acontece se não ajustarmos?**
Se você tiver apenas um construtor com parâmetros e nenhum construtor vazio (mesmo que privado), o EF Core vai disparar um erro ao tentar fazer um context.Veiculos.ToList(), dizendo que não encontrou um construtor adequado.

☑ **O Ajuste Profissional (Código)**

Código C#

```
// ITB.Domain/Entities/Veiculo.cs
public class Veiculo
{
    public int Id { get; private set; } // Ninguém de fora altera o Id
    public string Modelo { get; private set; }
    public string Placa { get; private set; }
    public int Ano { get; private set; }

    // Construtor Protegido para o Entity Framework
    protected Veiculo() { }

    // Única porta de entrada para criar um Veículo
    public Veiculo(string modelo, string placa, int ano)
    {
        // Regras de Integridade (Guards)
        if (string.IsNullOrWhiteSpace(modelo))
            throw new DomainException ("Modelo é obrigatório.");
```

```
        if (placa.Length != 7)
            throw new DomainException("Placa inválida.");

        if (ano < 1900 || ano > DateTime.Now.Year + 1)
            throw new DomainException("Ano do veículo fora da realidade.");

        Modelo = modelo;
        Placa = placa;
        Ano = ano;
    }

    // Comportamento: Alterar a placa requer validação
    public void AlterarPlaca(string novaPlaca)
    {
        if (string.IsNullOrWhiteSpace(novaPlaca)) throw new DomainException("Nova placa
inválida.");
        Placa = novaPlaca;
    }
}
```

🔍 **Comparação de Fluxos:**

Situação	Quem usa?	Qual caminho?
Criar um novo Veículo	O Desenvolvedor (Handler)	Usa o Construtor Público com Validação.
Ler do Banco (Select)	O Entity Framework	Usa o Construtor protected e preenche via Reflexão.
Alterar a Placa	O Desenvolvedor	É obrigado a usar o método AlterarPlaca() (Se existir).
Mudar a Placa por fora	Ninguém	O private set bloqueia o acesso direto.

Relacionamentos em Domínios Ricos (1:N)

OBJETIVO: Proteger a ligação entre Marca e Veículo.

☑ **Exemplo: Veículo Protegido (O Filho)**
O Veículo exige o ID do pai no nascimento.

Código C#

```
public class Veiculo
{
    public int Id { get; private set; }
    public string Modelo { get; private set; }
    public int MarcaId { get; private set; } // FK Protegida
    public virtual Marca Marca { get; private set; } // Navegação
```

```
    public Veiculo(string modelo, int marcaId)
    {
        if (marcaId <= 0) throw new DomainException("A Marca é obrigatória.");
        Modelo = modelo;
        MarcaId = marcaId;
    }
}
```

☑ **Exemplo: Marca com Coleção Encapsulada (O Pai)**
Impedimos que o programador limpe a lista de veículos por fora.

Código C#

```
public class Marca
{
    public int Id { get; private set; }
    public string Nome { get; private set; }

    private readonly List<Veiculo> _veiculos = new();

    // IReadOnlyCollection: Apenas leitura para quem está fora
    public virtual IReadOnlyCollection<Veiculo> Veiculos => _veiculos.AsReadOnly();

    public Marca(string nome) { Nome = nome; }
}
```

O "Cofre" (Backing Field):

Definida como **'private'** para que NINGUÉM fora desta classe possa manipular a lista.
O **'readonly'** garante que a instância da lista não seja substituída por outra.
O EF Core usará este campo via reflexão para preencher os dados vindo do banco.

```
    private readonly List<Veiculo> _veiculos = new();
```

A "Vitrine" (Propriedade de Navegação):

'virtual' permite que o EF Core faça Lazy Loading (carregamento sob demanda).
'IReadOnlyCollection' remove os métodos .Add() e .Remove() para quem está de fora.
'.AsReadOnly()' cria um invólucro (wrapper) que impede conversões maliciosas para List.

```
    public virtual IReadOnlyCollection<Veiculo> Veiculos => _veiculos.AsReadOnly();
```

🎯 **Regra de Ouro da Integridade:**
"Nunca permita que um objeto 'filho' (Veículo) seja criado sem o ID do seu 'pai' (Marca) se ele não puder existir sozinho. Isso evita **registros órfãos** no seu banco de dados!"